

TP 1 Jour 1

Liam Meridja

Question 1 – Onglet Jobs

Dans Jobs, on voit la liste de tous les jobs Spark lancés par l'application. Pour chaque job, on a son numéro (ID), une description (l'action qui l'a déclenché, comme count ou collect), l'heure de début, la durée, le nombre de stages et de tasks, et son état (réussi, en cours ou échoué). On a aussi une timeline des événements.

Question 2 – Les Stages

Un stage est une étape d'un job. Spark découpe un job en stages à chaque fois qu'il y a un *shuffle* (redistribution des données entre les machines), par exemple avec un groupBy ou un join

Question 3 – Les Tasks

Une task est la plus petite unité de travail. C'est le traitement d'une partition de données par un executor. S'il y a 8 partitions dans un stage, il y aura 8 tasks, qui peuvent tourner en parallèle.

Question 4 – Environment

Dans Environment, on voit la configuration de Spark : la version de Spark, la version de Java/Scala, les propriétés Spark (mémoire, nombre de cœurs, master, nom de l'application), les variables système et le classpath. C'est utile pour vérifier que la configuration est bien prise en compte.

Question 6

Un job est le travail que Spark lance quand on exécute une action comme count() ou show(). Un job est découpé en stages, qui sont des étapes séparées par un shuffle (quand les données sont redistribuées entre les partitions, par exemple avec un groupBy ou un join). Chaque stage est ensuite divisé en tasks, où une task traite une seule partition de données sur un executor. On a donc : une action déclenche un job, un job contient plusieurs stages, et un stage contient plusieurs tasks qui tournent en parallèle.

Question - combien de partitions

Mon DataFrame possède 32 partitions. C'est le nombre de cœurs disponibles pour Spark (local[*]), donc Spark crée une partition par cœur pour pouvoir traiter les données en parallèle.

Question 1 – Pourquoi Spark utilise-t-il des partitions ?

Spark utilise des partitions pour découper les données en petits morceaux. Comme ça, on peut traiter chaque morceau séparément et en même temps, sur plusieurs cœurs ou plusieurs machines, au lieu de tout traiter d'un seul bloc. Une partition correspond à une task.

Question 2 – Quel est l'intérêt d'avoir plusieurs partitions ?

L'intérêt, c'est le **parallélisme** : plusieurs tasks tournent en même temps, donc le traitement va plus vite. Ça permet aussi de gérer des données trop grosses pour tenir dans la mémoire d'une seule machine, et de bien répartir le travail entre les cœurs.

Question 3 – Est-ce que plus de partitions signifie toujours plus de performance ?

Non. Au début, ajouter des partitions améliore la performance parce qu'on utilise mieux les cœurs. Mais à partir d'un certain point, ça devient contre-productif : chaque partition a un coût (créer et gérer une task), et si on en a beaucoup trop, ce coût dépasse le gain. Il faut un bon équilibre selon la taille des données et le nombre de cœurs.

Question 4 – Que peut-il se passer si on a trop peu de partitions ?

- Certains cœurs ne font rien, car il n'y a pas assez de tasks pour les occuper : on gaspille des ressources.
- Chaque partition est très grosse, donc chaque task est longue et peut manquer de mémoire (erreur *out of memory*).
- Le traitement est plus lent car il n'y a pas assez de parallélisme.

Question 5 – Que peut-il se passer si on a énormément de partitions ?

- Il y a beaucoup de toutes petites tasks, et le temps passé à les créer et à les gérer (le *overhead*) devient plus grand que le travail utile.
- Le driver est surchargé parce qu'il doit suivre un très grand nombre de tasks.
- Le shuffle devient plus lourd, avec beaucoup de petits fichiers et d'échanges.
- Beaucoup de partitions vides, comme dans notre exemple avec 3 lignes sur 32 partitions, où les tasks ne servent presque à rien.

Question - Un nouveau Job est-il exécuté ?

Non pas de nouveau job exécuté

Question - Spark a-t-il déjà exécuté le traitement ?

toujours pas

Question 1 – Qu'est-ce que la Lazy Evaluation ?

C'est le fait que Spark ne calcule rien tout de suite. Quand on écrit des transformations (filter, select, groupBy...), Spark se contente de les noter dans un plan d'exécution. Le calcul ne démarre que lorsqu'on lance une action. C'est ce qu'on a vu dans la Spark UI : aucun job n'apparaissait tant qu'on n'avait pas fait de count() ou de show().

Question 2 – Pourquoi Spark utilise-t-il ce principe ?

Parce que Spark peut ainsi voir toutes les étapes avant de les exécuter, et optimiser le plan : regrouper des opérations, éviter des calculs inutiles, réduire les données lues. Ça évite aussi de gaspiller du temps et de la mémoire à calculer des résultats intermédiaires dont on n'a pas besoin.

Question 3 – Différence entre une transformation et une action

- Une transformation décrit une opération sur les données et renvoie un nouveau DataFrame. Elle n'est pas exécutée immédiatement (lazy).

- Une action demande un résultat (afficher, compter, récupérer, sauvegarder). Elle déclenche l'exécution de tout le plan et crée un job.

Question 4 – Deux exemples de transformations

- `select("age")`
- `filter(col("age") > 30)`

Question 5 – Deux exemples d'actions

- `show()`
- `count()`

Question 1 – Combien de Jobs a-t-on observés ?

On a observé 4 jobs pour un seul `df.show()`. Spark a d'abord lu 1 partition, puis 4, puis 20, puis les 7 restantes, parce qu'il cherchait 20 lignes alors qu'il n'y en avait que 3.

Question 2 – Combien de Stages sont associés au Job ?

Chaque job a 1 stage (1/1 dans la colonne Stages). Il n'y a aucun shuffle dans ce code, donc pas besoin de couper le travail en plusieurs stages.

Question 3 – Combien de Tasks ont été exécutées ?

Une task par partition lue : $1 + 4 + 20 + 7 = 32$ tasks au total. Ça correspond aux 32 partitions du DataFrame.

Question 4 – Relation entre Job, Stage et Task

Un job est composé d'un ou plusieurs stages, et chaque stage est composé de plusieurs tasks (une par partition).
Donc : 1 action → 1 ou plusieurs jobs → chaque job a un ou plusieurs stages → chaque stage a autant de tasks que de partitions.

Question 5 – Que se passe-t-il lorsqu'une Action est exécutée ?

L'action déclenche le calcul. Spark regarde tout le plan des transformations, l'optimise, le découpe en jobs, stages et tasks, puis envoie les tasks aux executors qui les exécutent en parallèle. Le résultat est ensuite renvoyé au driver, et on le voit dans la Spark UI.

Question 6 – Pourquoi Spark découpe-t-il le travail en Tasks ?

Pour pouvoir traiter les données en parallèle. Chaque task s'occupe d'une partition, et plusieurs tasks peuvent tourner en même temps sur différents cœurs ou machines. Ça rend le traitement plus rapide, et si une task échoue, Spark peut la relancer seule sans tout refaire.

Question : Que se passe-t-il dans Spark entre l'écriture de `df.filter(...)` et l'exécution de `df.show()` ?

Spark fonctionne en **lazy evaluation** : il distingue les transformations, qui décrivent un calcul, des actions, qui le déclenchent.

Quand on écrit `df.filter(...)`, il s'agit d'une transformation, donc rien n'est exécuté. Spark ajoute simplement une étape Filter au plan logique et retourne un nouveau DataFrame, qui n'est qu'une description du calcul à effectuer.

Quand on appelle `df.show()`, il s'agit d'une action, et c'est elle qui déclenche l'exécution. Le moteur Catalyst commence par analyser le plan puis l'optimiser, par exemple avec le predicate pushdown et le column pruning. Spark génère ensuite le plan physique le moins coûteux. Le DAG Scheduler crée alors un job, qu'il découpe en stages aux frontières de shuffle, puis en tasks, à raison d'une par partition. Enfin, les executors exécutent ces tasks et renvoient le résultat au driver, qui affiche les 20 premières lignes.